

API Monitoring Project Wiki

English · [繁體中文](#)

API Monitoring — Hybrid Storage · Project Wiki

In one sentence: A backend monitoring demo system featuring a Cache-Aside caching strategy, chaos-engineering fault simulation, and full observability.

Table of Contents

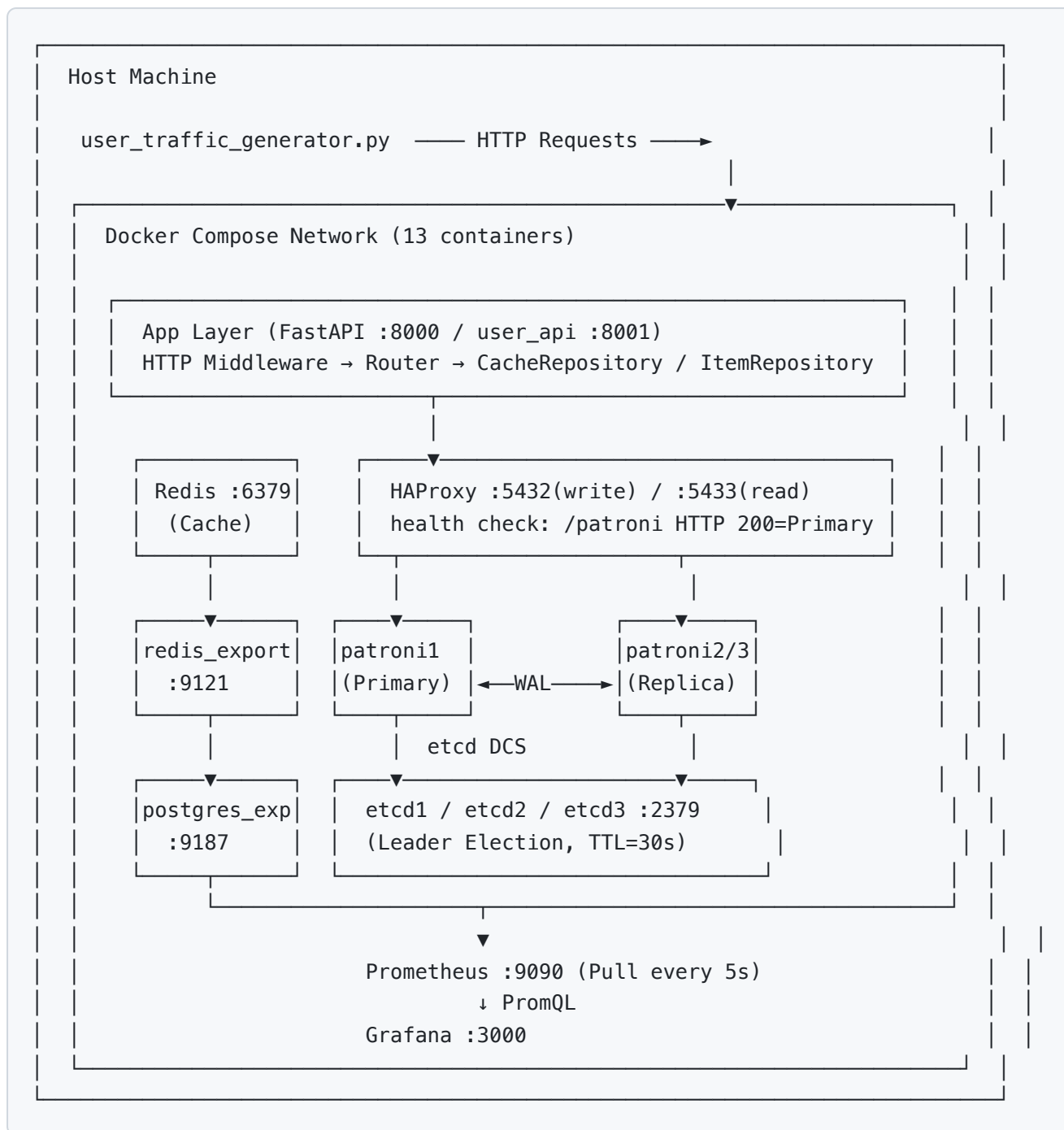
1. [Tech Stack & Services](#)
 2. [System Architecture](#)
 3. [Repository Structure](#)
 4. [Component Deep-Dive](#)
 - [4.1 config.py — Configuration Layer](#)
 - [4.2 metrics.py — Metrics Layer](#)
 - [4.3 CacheRepository — Redis Wrapper](#)
 - [4.4 ItemRepository — PostgreSQL Wrapper](#)
 - [4.5 routers/items.py — Business Routes](#)
 - [4.6 routers/anomaly.py — Chaos Routes](#)
 - [4.7 main.py — App Factory](#)
 5. [Data Flows](#)
 - [5.1 Cache-Aside Read Flow](#)
 - [5.2 Metrics Scrape Flow](#)
 6. [API Reference](#)
 7. [Prometheus Metrics Reference](#)
 8. [Chaos Engineering Scenarios](#)
 9. [OOP Design Decisions](#)
 10. [How to Run](#)
-

1. Tech Stack & Services

Category	Technology	Version	Port
HTTP Framework	FastAPI + Uvicorn	0.103.1 / 0.23.2	8000
User Service	user_api.py (FastAPI + Uvicorn)	runs directly on host	8001
Relational DB (HA)	PostgreSQL 15–bookworm via Patroni 4.1.3	3 nodes	HAProxy :5432
HA Proxy	HAProxy 2.8	write:5432 / read:5433 / stats:7001	5432/5433/7001
DCS	etcd 3.5.16	3 nodes (Raft Consensus), :2379/:2380	2379/2380
Cache (HA)	Redis 7–alpine — Sentinel HA	1 Master + 2 Replicas + 3 Sentinels	6379/6380/6381/26379
Metrics Scraper	Prometheus	latest	9090
Visualization	Grafana	13.0.1	3000
Redis Exporter	oliver006/redis_exporter	latest	9121
PostgreSQL Exporter	prometheuscommunity/postgres-exporter	latest	9187
Container Runtime	Docker Compose	3.8	—

18 Docker containers in total: etcd1/2/3, patroni1/2/3, haproxy, api, redis-master, redis-replica1, redis-replica2, sentinel1/2/3, prometheus, grafana, redis_exporter, postgres_exporter

2. System Architecture



Four data flows: – **Main App Business Flow:** `user_traffic_generator` → FastAPI (8000) → Redis / HAProxy(5432) → Patroni Primary – **User API Business Flow:** `user_traffic_generator` → `user_api.py` (8001) → Redis / HAProxy(5432) → Patroni Primary – **Metrics Scrape Flow:** Prometheus actively pulls → FastAPI (8000) `/metrics` + `user_api` (8001) `/metrics` + the two exporters – **Visualization Flow:** Grafana sends PromQL to Prometheus → renders charts – **HA Failover Flow:** Patroni elects a leader automatically via the etcd DCS (RTO=5s) → HAProxy re-routes automatically

3. Repository Structure

```

api_monitoring/
|
├── docker-compose.yml      ← Defines and starts all 13 containers (incl. HA
cluster)
├── user_api.py             ← Second FastAPI service (runs on host, port 8001)
                             Simulates user behavior: search/query/register
customers
|
                             Includes chaos endpoints: /chaos/log-
storm/start|stop|status
|
                             Exposes user_api_* metrics for Prometheus to
scrape
├── user_traffic_generator.py ← Dual-service traffic generator + chaos control
                             Hits user_api (8001) + main app (8000)
simultaneously
|
                             Six-phase interactive demo: --interactive
                             Drives data into every Grafana panel
├── logger_manager.py       ← Generic logging utility (used by user_api.py)
                             setup_logging(name) + @log_decorator
├── traffic_generator.py    ← (legacy, replaced by user_traffic_generator.py)
├── chaos_scenario.py       ← (legacy, replaced by user_traffic_generator.py -
-interactive)
|
├── prometheus/
|   └── prometheus.yml      ← Prometheus scrape job config
                             4 jobs: fastapi-app / redis / postgres / user-
api
|
├── patroni/
|   ├── patroni1.yml        ← Patroni cluster config
|   ├── patroni2.yml        ← patroni1 config (scope: pg-ha-cluster)
|   └── patroni3.yml        ← patroni2 config
                             ← patroni3 config
|
├── haproxy/
|   └── haproxy.cfg         ← HAProxy routing config
                             pg_write frontend :5432 → Primary (/patroni
HTTP 200)
                             pg_read frontend :5433 → Replicas (/patroni
HTTP 503)
|
├── app/
image)
|   ├── Dockerfile
|   ├── requirements.txt
|   |
|   ├── config.py           ← Settings (frozen dataclass)
|   ├── metrics.py          ← AppMetrics (Singleton, all Prometheus metrics)
|   ├── main.py             ← App Factory (assembly entry point)
|   |
|   └── repositories/       ← Data-access layer (DAL)

```

├──	cache_repo.py	← CacheRepository (wraps Redis)
├──	item_repo.py	← ItemRepository (wraps PostgreSQL)
├──	routes/	← HTTP routing layer
├──	items.py	← /api/data, /api/write
├──	anomaly.py	← /api/anomaly/* (10 chaos endpoints)
├──	document/	
├──	dir_structure.txt	← Project directory notes
├──	system_architecture.mmd	← System architecture diagram (Mermaid) with HA cluster
├──	grafana_panel_labels_guide.md	← Grafana panel legend label reference (19 panels)
├──	project_wiki.md	← This document
├──	usage.txt	← Startup steps

4. Component Deep-Dive

4.1 `config.py` — Configuration Layer

Design pattern: Immutable Value Object (frozen dataclass) + Factory Method

```
@dataclass(frozen=True)
class Settings:
    redis_sentinel_hosts: str # Sentinel connection list
    redis_master_name: str   # The master name Sentinel monitors
    database_url: str
    cache_ttl: int           # Redis TTL, default 10 seconds

    @classmethod
    def from_env(cls) -> "Settings":
        ...

settings = Settings.from_env() # Module-level singleton
```

Key design points: – `frozen=True`: The settings object cannot be modified by any code at runtime, preventing accidental state changes – `from_env()` Factory: Reads uniformly from environment variables, easy to replace with fixed values in tests, independent of the system environment – `CACHE_TTL` externalized: The cache expiry can be tuned via the `environment` section of `docker-compose.yml` without changing code

Environment variable mapping:

	Environment variable	Default	Description
—	<code>REDIS_SENTINEL_HOSTS</code>	<code>sentinel1:26379,sentinel2:26379,sentinel3:26379</code>	Sentinel node list
	<code>REDIS_MASTER_NAME</code>	<code>mymaster</code>	The master name Sentinel monitors

`DATABASE_URL` | `postgresql://appuser:...` | PostgreSQL connection string | `CACHE_TTL` | `10`
| Redis key TTL in seconds |

4.2 `metrics.py` — Metrics Layer

Design pattern: Singleton (overriding `__new__`)

```
class AppMetrics:
    _instance: "AppMetrics | None" = None

    def __new__(cls) -> "AppMetrics":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._register_metrics()
        return cls._instance
```

Why a Singleton: A limitation of `prometheus_client` is that a metric of the same name can only be registered once per Python process. No matter which module calls `AppMetrics()`, it returns the same instance, ensuring metrics are never created twice.

All defined metrics:

Metric name	Type	Labels	Purpose
<code>http_requests_total</code>	Counter	<code>method</code> , <code>endpoint</code> , <code>http_status</code>	API request count, used to compute error rate
<code>http_request_duration_seconds</code>	Histogram	<code>method</code> , <code>endpoint</code>	Response-time distribution, used to compute P99
<code>redis_cache_hits_total</code>	Counter	—	Redis hit count
<code>redis_cache_misses_total</code>	Counter	—	Redis miss count (falls back to DB)
<code>redis_errors_total</code>	Counter	—	Redis connection or operation error count
<code>db_errors_total</code>	Counter	<code>operation</code>	DB operation failure count
<code>db_query_duration_seconds</code>	Histogram	<code>query_type</code>	DB query duration distribution

Counter vs Histogram: – **Counter:** A monotonically increasing cumulative number; use `rate()` to convert to a per-second rate – **Histogram:** Distributes observed values into multiple buckets; use `histogram_quantile()` to compute percentiles (P50, P99)

4.3 `CacheRepository` — Redis Sentinel Wrapper

Responsibility: The single entry point for all Redis Sentinel operations, including automatic master discovery, metric tracking, and error handling.

```

CacheRepository
|
|— __init__(sentinel_hosts, master_name) ← Sentinel connection init
|— ping() ← Startup health check
|— get(key) → Optional[str] ← Read cache (auto-updates hit/miss counters)
|— setex(key, ttl, value) ← Write cache (silently degrades on failure)
|— flush() ← Flush the Redis DB (for chaos)
|— simulate_connection_failure() ← Connect to a dead node (for chaos)

```

Sentinel initialization logic:

```

from redis.sentinel import Sentinel

def __init__(self, sentinel_hosts: str, master_name: str) -> None:
    hosts = [(h.split(":")[0], int(h.split(":")[1])) for h in
              sentinel_hosts.split(",")]
    _sentinel = Sentinel(hosts, socket_timeout=0.5, socket_connect_timeout=0.5)
    self._client = _sentinel.master_for(master_name, decode_responses=True)

```

Failover behavior: – Sentinel detects the master is down (`down-after-milliseconds=5000`) – quorum=2 votes elect a new master (`failover-timeout=10000`) – The redis-py `Sentinel` client automatically points to the new master on the next `master_for()` call

Graceful degradation strategy: – An error in `get()` → returns `None` (equivalent to a cache miss) + `redis_errors` +1, the main flow continues to the DB – An error in `setex()` → silently logged, not raised, does not affect the HTTP response – An error in `flush()` → raised, the Router decides whether to return 503

This design ensures that a **Redis outage does not bring down the API as a whole** — it only degrades performance (all requests hit the DB).

4.4 ItemRepository — PostgreSQL Wrapper

Responsibility: The single entry point for all PostgreSQL operations, including connection-pool management, schema initialization, CRUD, and chaos operations.

```

ItemRepository
|
|— initialize_schema() ← Create tables + insert 100 seed rows
(idempotent)
|— get_random_item() → dict ← SELECT RANDOM() LIMIT 1
|— insert_item(name, value) → float ← INSERT, returns duration
|— execute_slow_query() → float ← SELECT pg_sleep(3) (for chaos)
|— dispose() ← Release the connection pool

```


Connection-pool config (SQLAlchemy):

```
create_engine(  
    database_url,  
    pool_pre_ping=True,    # PING before borrowing a connection, auto-reconnect  
                           (avoids dead idle connections)  
    pool_size=5,          # Maintain up to 5 persistent connections  
)
```

Schema:

```
CREATE TABLE IF NOT EXISTS items (  
    id          SERIAL PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    value       FLOAT        NOT NULL,  
    created_at  TIMESTAMP    DEFAULT NOW()  
);
```

`initialize_schema()` is **idempotent**: `IF NOT EXISTS` ensures repeated calls do not error, and `COUNT(*)` ensures data is only seeded when the table is empty.

4.5 `routers/items.py` — Business Routes

Prefix: `/api`

Dependency injection: Repository instances are obtained from `request.app.state` (attached at startup by `lifespan`).

```
cache_repo = request.app.state.cache_repo  
item_repo  = request.app.state.item_repo
```

This makes the Router entirely independent of global variables; the Repository implementation can be swapped (e.g., for a Mock in tests) without changing Router code.

Full `GET /api/data` logic:

```

1. cache_repo.get("data:latest")
   |
   |— Not None (Hit) → return {"source": "cache (Redis)", ...}
   |
   |— None (Miss)
       ↓
       item_repo.get_random_item()
       ↓
       cache_repo.setex("data:latest", TTL, result)
       ↓
       return {"source": "database (PostgreSQL)", ...}

```

4.6 routers/anomaly.py — Chaos Routes

Prefix: `/api/anomaly`

Each endpoint maps to a real-world failure scenario:

Endpoint	Simulated scenario	Implementation mechanism
<code>/lag</code>	The whole system slows down	<code>time.sleep(2.5)</code>
<code>/error</code>	The application throws an exception	<code>Response(status_code=500)</code>
<code>/db-overload</code>	DB slow query, missing index	<code>SELECT pg_sleep(3)</code>
<code>/cache-flush</code>	Cache hot restart, stampede	<code>redis.flushdb()</code>
<code>/redis-down</code>	Redis node unreachable	Connects to <code>non-existent-redis:6379</code>

4.7 main.py — App Factory

Responsibility: The single assembly point, containing no business logic.

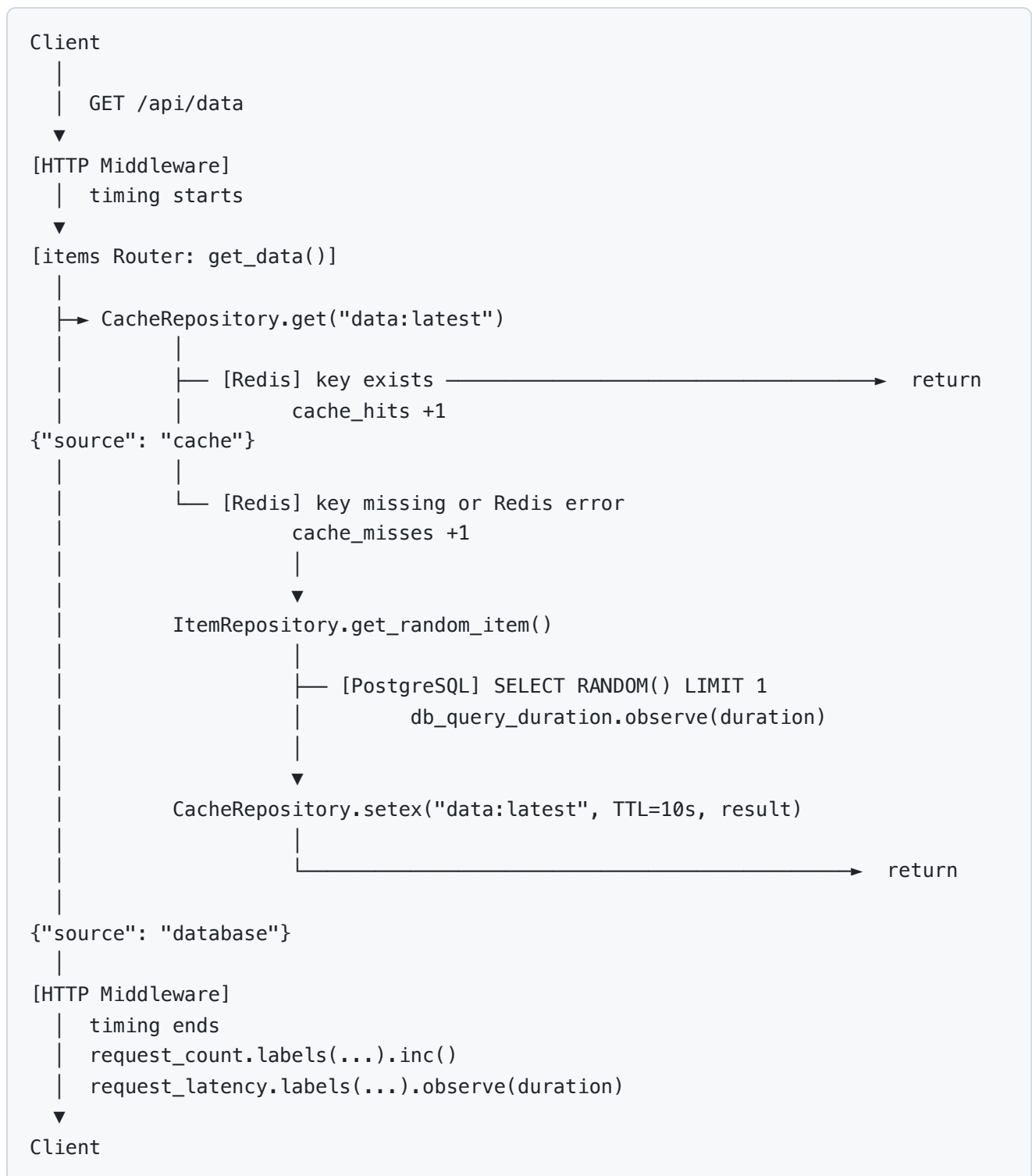
```
lifespan (startup)
  └─ CacheRepository(settings.redis_sentinel_hosts, settings.redis_master_name)
→ app.state.cache_repo
  └─ ItemRepository(settings.database_url) → app.state.item_repo
    └─ initialize_schema()

FastAPI app
  └─ Middleware: monitor_requests (timing + writing metrics)
  └─ Mount: /metrics → Prometheus ASGI app
  └─ include_router: items_router (/api/*)
  └─ include_router: anomaly_router (/api/anomaly/*)

lifespan (shutdown)
  └─ item_repo.dispose()
```

5. Data Flows

5.1 Cache-Aside Read Flow



5.2 Metrics Scrape Flow

Prometheus (runs once every 5 seconds)

- └ GET http://api:8000/metrics
 - http_requests_total
 - http_request_duration_seconds
 - redis_cache_hits_total
 - redis_cache_misses_total
 - redis_errors_total
 - db_errors_total
 - db_query_duration_seconds
- └ GET http://redis_exporter:9121/metrics
 - redis_connected_clients
 - redis_memory_used_bytes
 - redis_commands_processed_total
 - ... (50+ low-level Redis metrics)
- └ GET http://postgres_exporter:9187/metrics
 - pg_stat_activity_count
 - pg_stat_database_tup_fetched
 - pg_locks_count
 - ... (100+ low-level PostgreSQL metrics)

Grafana

- └ PromQL queries → Prometheus → renders the Dashboard

6. API Reference

Business endpoints

Method	Path	Description	Response example
GET	/	Health check	<pre>{"message": "API is running normally."}</pre>
GET	/api/data	Cache-Aside read	<pre>{"source": "cache (Redis)", "data": "..."} </pre>
GET	/api/write	Write random data	<pre>{"status": "written", "name": "item_1234", "value": 567.89} </pre>
GET	/metrics	Prometheus metrics endpoint	Plain text (Prometheus exposition format)

Chaos endpoints

Method	Path	Expected behavior	HTTP Status
GET	/api/anomaly/lag	Responds after a 2.5s delay	200
GET	/api/anomaly/error	Returns an error immediately	500
GET	/api/anomaly/db-overload	Responds after a 3s wait (DB slow query)	200
GET	/api/anomaly/cache-flush	Flushes Redis, responds immediately	200
GET	/api/anomaly/redis-down	Triggers a Redis connection failure	503

7. Prometheus Metrics Reference

Key PromQL queries

API error rate (5xx)

```
sum(rate(http_requests_total{http_status=~"5.."}[1m]))
/
sum(rate(http_requests_total[1m]))
```

API P99 response time

```
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[1m])) by (le, endpoint)
)
```

Redis cache hit rate

```
rate(redis_cache_hits_total[1m])
/
(rate(redis_cache_hits_total[1m]) + rate(redis_cache_misses_total[1m]))
```

Redis error rate

```
rate(redis_errors_total[1m])
```

DB slow query P99

```
histogram_quantile(0.99,
  sum(rate(db_query_duration_seconds_bucket[1m])) by (le, query_type)
)
```

Request rate per status code

```
sum by (http_status) (rate(http_requests_total[1m]))
```

8. Chaos Engineering Scenarios

Scenario 1: Cache Stampede

Trigger: `GET /api/anomaly/cache-flush`

Mechanism: 1. The Redis cache is flushed, all `data:latest` keys disappear 2. The next batch of `/api/data` requests are all cache misses 3. All requests hit PostgreSQL at once (traffic spike) 4. After 10 seconds the TTL rebuilds, and the hit rate slowly recovers

Metrics to observe: – `redis_cache_misses_total` spikes – `db_query_duration_seconds` P99 rises briefly (increased DB pressure) – Hit Rate first drops to 0% then slowly recovers

Scenario 2: Database Slow Query

Trigger: `GET /api/anomaly/db-overload`

Mechanism: – Runs `SELECT pg_sleep(3)`, forcing PostgreSQL to hold a connection for 3 seconds – Simulates real-world scenarios like missing indexes, lock contention, or large full-table scans

Metrics to observe: – `db_query_duration_seconds{query_type="slow_query"}` P99 spikes to ~3s – Overall `http_request_duration_seconds` P99 rises as well

Scenario 3: Redis Connection Failure

Trigger: `GET /api/anomaly/redis-down`

Mechanism: – `CacheRepository.simulate_connection_failure()` attempts to connect to the non-existent `non-existent-redis:6379` – `socket_connect_timeout=1` ensures an exception is thrown within 1 second – `redis_errors_total` increments – Actual `/api/data` traffic is unaffected (it reads from the normal Redis); this endpoint only increments the error counter

Metrics to observe: – `redis_errors_total` keeps accumulating

Scenario 4: HTTP 500 Error (application crash)

Trigger: `GET /api/anomaly/error`

Metrics to observe: – `http_requests_total{http_status="500"}` increases – Error Rate (5xx %) rises

9. OOP Design Decisions

Why use a layered architecture?

Router Layer	← Only handles the HTTP protocol (request/response)
↓	
Repository Layer	← Only handles data access (SQL / Redis commands)
↓	
Infrastructure	← Only handles connections (SQLAlchemy Engine / Redis Client)

Benefits: – **Single responsibility:** Each class has only one reason to change – **Testability:** The Router can be unit-tested with a Mock Repository – **Replaceability:** Swapping Redis for Memcached later only requires changing `CacheRepository`, with zero Router changes

Why does the Repository update metrics itself?

When the Router calls `cache_repo.get(key)`, it does not need to know that the metric `redis_cache_hits_total` exists; the metric logic is fully encapsulated within the Repository. This follows the **Tell, Don't Ask** principle.

Why use `app.state` for dependency injection instead of global variables?

```
# ❌ Old approach: global variable, hard to test, risk of race conditions
redis_conn = None

# ✅ New approach: injected via app.state, with a clear lifecycle
app.state.cache_repo = CacheRepository(...)
```

`app.state` has the same lifecycle as the FastAPI app, managed uniformly by `lifespan`, avoiding the “object created but connection not yet initialized” problem.

10. How to Run

Start all services

```
cd api_monitoring
docker-compose up -d --build
```

Verify each service

```
# API health check
curl http://localhost:8000/

# Prometheus metrics (confirm instrumentation works)
curl http://localhost:8000/metrics | grep http_requests

# Prometheus Web UI
open http://localhost:9090

# Grafana Dashboard (credentials: admin / admin)
open http://localhost:3000
```

Generate traffic

```
python traffic_generator.py
```

Manually trigger anomaly scenarios

```
# Cache Stampede (watch the Cache Hit Rate plunge)
curl http://localhost:8000/api/anomaly/cache-flush

# DB slow query (watch DB P99 spike to 3s)
curl http://localhost:8000/api/anomaly/db-overload

# Redis disconnect (watch redis_errors_total accumulate)
curl http://localhost:8000/api/anomaly/redis-down
```

Stop services

```
# Stop but keep containers (data persists)
docker-compose stop

# Stop and remove containers (data is lost)
docker-compose down
```

View the API container logs

```
docker-compose logs -f api
```